

Shared Velocity-Input Utilities Specification

Uniform Sampling, Atom Selection, Weighting, Drift Removal, and Per-Atom Output Mapping

mdstats

2026-07-22

Contents

1 Purpose and implementation status	1
2 Motive	2
3 Provenance and attribution boundary	2
3.1 External methods	2
3.2 mdstats design	2
4 Module dependencies	2
5 Type aliases	3
6 Resolved data structure	3
6.1 Array meanings	3
7 Function specifications	4
7.1 validate_uniform_time_grid	4
7.2 resolve_velocity_weights	4
7.3 compute_drift_velocity	5
7.4 resolve_per_atom_output	5
7.5 prepare_velocity_inputs	6
8 VACF integration	6
9 Complexity and memory	6
10 Edge cases	7
11 Required tests	7
12 Acceptance criteria	7
13 H0 semantic-signature and immutability integration	7

1 Purpose and implementation status

This document specifies the private module
mdstats/analysis/_velocity_common.py

implemented in `mdstats 0.19.6a0` as roadmap stage **VC0**.

The module centralizes input preparation shared by velocity-based self observables. Its consumers are `compute_vacf()`, the direct Welch velocity-spectrum estimator, and `compute_charge_current()`. Collective current construction reuses only trajectory validation, atom selection, drift construction, and the semantic signature; it does not reuse self-correlation weights or per-atom output mapping because its charge algebra is distinct.

The module performs no VACF, Fourier transform, periodogram, transport integration, or physical interpretation.

2 Motive

Before VC0, `vacf.py` owned four private operations:

1. uniform-time-grid validation;
2. atom-weight resolution;
3. framewise center-of-mass or center-of-geometry drift construction;
4. canonical/local mapping for per-atom output.

Copying those operations into the direct velocity-spectrum estimator would create two independently evolving definitions of the same measured atoms and velocities. VC0 extracts the validated behavior once and makes estimator code consume one resolved bundle.

The refactor must preserve all existing VACF numerical results, warnings, metadata, selector order, and exceptions.

3 Provenance and attribution boundary

3.1 External methods

VC0 introduces **no borrowed mathematical algorithm**. Uniform-grid checks, weighted averages, index validation, and dictionary-style index lookup are standard programming and numerical-analysis operations that do not require a specific algorithmic attribution.

Chemical-symbol selection remains delegated to `selection.py`, whose symbol lookup uses the Atomic Simulation Environment data table. VC0 neither changes nor reimplements that dependency.

3.2 mdstats design

The following are package-specific decisions:

- one common preparation layer is shared by VACF, direct velocity spectra, and collective charge-current construction;
- the full trajectory velocity array is retained without a large copy;
- small selection, weight, drift, and output-mapping arrays are owned by the returned bundle;
- physical framewise drift removal is separated from later segment detrending;
- measured and drift selections use canonical atom indices;
- per-atom output preserves user request order while also storing local measured-selection indices;
- a drift reference equal to a strict measured subset is reported as a flag so each public estimator can emit its own warning category;
- all first-generation velocity estimators require uniform sampling.

4 Module dependencies

AtomisticFrameCollection
|

```

    +--> trajectory/time/velocity validation
    |
selection.resolve_atom_selection
    |
    +--> measured selection
    +--> drift selection
    |
_velocity_common.py
    |
    +--> VACF
    +--> direct Welch velocity spectrum
    +--> collective charge-current construction

```

The module depends on:

- `mdstats.collection.AtomisticFrameCollection`;
- `mdstats.analysis.selection.resolve_atom_selection`;
- NumPy arrays and reductions.

It is private and is not exported from `mdstats` or `mdstats.analysis`.

5 Type aliases

```

FloatArray = NDArray[np.float64]
IntArray = NDArray[np.int64]
DriftMode = Literal["center_of_mass", "center_of_geometry"]
WeightInput = Literal["uniform", "mass"] | ArrayLike

```

6 Resolved data structure

```

@dataclass(frozen=True, slots=True)
class VelocityInputBundle:
    sample_spacing_ps: float
    velocities: NDArray[np.float64]           # (T, N, 3)

    atom_indices: NDArray[np.int64]          # (M,)
    atom_weights: NDArray[np.float64]        # (M,)
    weight_sum: float
    weighting: str
    weight_units: str
    correlation_units: str

    drift_mode: str | None
    drift_atom_indices: NDArray[np.int64] | None
    drift_velocity: NDArray[np.float64] | None # (T, 3)
    drift_matches_measured_subset: bool

    per_atom_indices: NDArray[np.int64] | None
    per_atom_local_indices: NDArray[np.int64] | None

    signature: DynamicsInputSignature

```

6.1 Array meanings

Let the full collection contain T frames and N atoms. Let the measured selection contain M atoms.

- `velocities[t, i, alpha]` is the canonical Cartesian velocity in Angstrom/ps.

- `atom_indices[j]` maps measured-local atom j to canonical atom numbering.
- `atom_weights[j]` is the nonnegative self-correlation or self-periodogram weight of measured-local atom j .
- `per_atom_indices[q]` is a requested canonical atom index.
- `per_atom_local_indices[q]` is the corresponding location inside `atom_indices`.

The two per-atom arrays satisfy

```
atom_indices[per_atom_local_indices] == per_atom_indices
```

in exactly the user's requested order.

7 Function specifications

7.1 validate_uniform_time_grid

```
def validate_uniform_time_grid(
    collection: AtomisticFrameCollection,
    *,
    analysis_name: str,
) -> float:
    ...
```

7.1.1 Contract

The collection must:

- have trajectory semantics;
- contain at least two frames;
- have a complete physical time axis;
- have finite strictly increasing times;
- satisfy uniform sampling within `rtol=1e-10`, `atol=1e-14` in ps.

The function returns the first time increment as `float`.

No resampling, interpolation, frame dropping, or time-axis repair is performed. The diagnostic uses `analysis_name` so the same helper produces estimator-specific messages.

7.2 resolve_velocity_weights

```
def resolve_velocity_weights(
    collection: AtomisticFrameCollection,
    selected: NDArray[np.int64],
    weights: WeightInput,
) -> tuple[FloatArray, str, str, str]:
    ...
```

7.2.1 Supported modes

Input	Resolved weights	Weight units	Correlation-density units
"uniform"	$q_i = 1$	dimensionless	Angstrom-squared/ps-squared
"mass"	$q_i = m_i$	amu	amu Angstrom-squared/ps-squared
explicit array	user values	dimensionless	Angstrom-squared/ps-squared

Explicit weights must:

- have shape (M,);
- be finite;
- be nonnegative;
- contain at least one strictly positive value.

The returned array is owned by the result and does not alias an explicit user array.

The helper resolves weights only. Whether a downstream physical observable permits mass or nonuniform weights remains the downstream module’s responsibility.

7.3 compute_drift_velocity

```
def compute_drift_velocity(
    collection: AtomisticFrameCollection,
    velocities: FloatArray,
    drift_indices: IntArray,
    *,
    drift_mode: DriftMode,
) -> FloatArray:
    ...
```

For center-of-geometry subtraction,

$$\mathbf{v}_{\text{COG}}(t) = \frac{1}{N_d} \sum_{i \in d} \mathbf{v}_i(t).$$

For center-of-mass subtraction,

$$\mathbf{v}_{\text{COM}}(t) = \frac{\sum_{i \in d} m_i \mathbf{v}_i(t)}{\sum_{i \in d} m_i}.$$

The mass denominator must be finite and strictly positive. The function returns shape (T, 3) and does not modify the collection velocity array.

Physical drift subtraction is a framewise coordinate choice. It is distinct from Welch segment-mean detrending, which acts separately on each finite segment and changes low-frequency spectral content.

7.4 resolve_per_atom_output

```
def resolve_per_atom_output(
    selected: IntArray,
    per_atom: bool,
    per_atom_indices: ArrayLike | None,
    n_atoms: int,
) -> tuple[IntArray | None, IntArray | None]:
    ...
```

Behavior:

- if neither form of per-atom output is requested, return (None, None);
- if per_atom=True and no explicit list is supplied, return all measured atoms in measured-selection order;
- if an explicit list is supplied, preserve its order;
- require one-dimensional integer indices;
- reject empty, duplicate, negative, out-of-range, or unmeasured atoms;
- return both canonical and measured-local indices.

The local-index lookup is $O(M + Q)$ for M measured atoms and Q requested outputs.

7.5 prepare_velocity_inputs

```
def prepare_velocity_inputs(
    collection: AtomisticFrameCollection,
    *,
    analysis_name: str,
    species: SpeciesSelection = None,
    atom_indices: ArrayLike | None = None,
    weights: WeightInput = "uniform",
    drift_mode: DriftMode | None = None,
    drift_species: SpeciesSelection = None,
    drift_atom_indices: ArrayLike | None = None,
    per_atom: bool = False,
    per_atom_indices: ArrayLike | None = None,
) -> VelocityInputBundle:
    ...
```

7.5.1 Algorithm

1. Validate the collection type and nonempty analysis label.
2. Validate trajectory semantics and uniform time sampling.
3. require the complete Cartesian velocity field.
4. Resolve the measured atom selection.
5. Validate the drift mode and drift-selection consistency.
6. Resolve the drift selection when enabled.
7. Set `drift_matches_measured_subset` when the drift reference equals the measured selection and the measured selection excludes at least one collection atom.
8. Compute the framewise drift velocity.
9. Resolve atom weights and units.
10. Resolve per-atom canonical/local output maps.
11. Return one immutable bundle.

7.5.2 Warning boundary

This private function does not emit estimator-specific warnings. For example, `compute_vacf()` converts `drift_matches_measured_subset=True` into `CollectiveMotionVACFWarning`. A direct spectrum estimator may later use a spectral warning category while preserving the same detection rule.

8 VACF integration

`compute_vacf()` now calls `prepare_velocity_inputs(..., analysis_name="VACF")` and then uses the resolved bundle for both direct and FFT backends.

This is a behavioral refactor only. Required regression invariants include:

- direct and FFT VACF numerical arrays are unchanged;
- selected atom order is unchanged;
- explicit per-atom request order is unchanged;
- weight units and correlation units are unchanged;
- center-of-mass and center-of-geometry corrections are unchanged;
- strict-subset drift warnings are unchanged;
- all prior validation failures remain failures.

9 Complexity and memory

Uniform-grid validation is $O(T)$. Selection and weight preparation are $O(N + M)$. Drift construction is $O(TN_d)$ for N_d reference atoms. Per-atom mapping is $O(M + Q)$.

The bundle intentionally avoids copying the full (T, N, 3) velocity array. Only the optional drift array has trajectory-sized storage, (T, 3). Selection, weight, and index arrays scale with the selected atom count.

10 Edge cases

- A trajectory with nonuniform times is rejected even if deviations are small enough to look visually regular but exceed the numerical tolerance.
- A drift selection without a drift mode is rejected.
- An invalid drift mode is rejected before an estimator begins numerical work.
- A mass-weighted drift reference with invalid total mass is rejected.
- Boolean per-atom arrays are not accepted as integer index lists.
- A zero explicit weight for some atoms is allowed; all-zero weights are not.
- The drift reference may include atoms outside the measured selection.
- The measured selection may equal all atoms; in that case equality with the drift reference does not trigger the strict-subset collective-motion flag.

11 Required tests

1. measured species/index selection and canonical ordering;
2. uniform, mass, and explicit weight modes;
3. explicit weight copying and invalid weight rejection;
4. center-of-mass versus center-of-geometry values;
5. drift-reference strict-subset flag;
6. canonical/local per-atom mapping and request order;
7. duplicate, out-of-range, and unmeasured per-atom rejection;
8. trajectory, velocity, and uniform-time guards;
9. inconsistent drift arguments;
10. complete regression of the pre-refactor VACF test suite.

12 Acceptance criteria

VC0 is complete when:

- `_velocity_common.py` contains the normative shared helpers;
- `vacf.py` contains no duplicate input-resolution implementation;
- existing VACF tests pass unchanged;
- focused helper tests pass;
- the module remains private;
- the Markdown and PDF specifications agree;
- the architecture roadmap marks VC0 complete.

13 H0 semantic-signature and immutability integration

`prepare_velocity_inputs()` constructs a complete `DynamicsInputSignature` from the normalized collection and the exact measured/drift selections. The velocity bundle stores the source three-dimensional subspace; later projected consumers use `signature.with_subspace(...)`.

The signature records source format/files, a deterministic normalized-trajectory fingerprint, exact frame IDs and times, frame count and sample spacing, measured atoms, laboratory-coordinate semantics, exact drift-reference atoms, velocity source, and the full Cartesian basis.

The bundle intentionally retains the normalized collection velocity array to avoid copying the complete trajectory. Its small selection, weighting, drift, and signature arrays are owned and read-only. Downstream public results

never alias the retained velocity array. Integer stride/block controls reject booleans, and `per_atom` requires an actual boolean. The authoritative common contract is `docs/specs/analysis/_dynamics_common_spec.md`.

Required H0 regressions verify signature propagation through `compute_vacf()` and direct Welch spectra, exact drift-reference identity, immutable selection arrays, and strict boolean validation.